



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

Name: Abobakr Mohamed

Elameen

PRN: 25070122015

Branch: CSE (2025-29)

Semester: 3

Subject: Programming with Java

ASSIGNMENT No.: 6

TITLE: : Write a Java program to demonstrate the use of an inner class, an anonymous class.

Problem Statement:

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives:

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory:

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling.

Outcomes:

The inner class was successfully used to access private members of the outer class, demonstrating tight encapsulation. The anonymous class was used to provide a one-time implementation of the Greet interface without creating a separate named class. This showed how inner and anonymous classes reduce code overhead for tasks that don't need reusable class definitions.

```
class Outer {
    private String message = "Hello from Outer class";

    class Inner {
        void display() {
            System.out.println("Inner class accessing: " + message);
        }
    }
}

interface Greet {
    void sayHello();
}

public class JAVA_EXP_6 {
    public static void main(String[] args) {

        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        inner.display();

        Greet g = new Greet() {
            public void sayHello() {
                System.out.println("Hello from Anonymous class");
            }
        };
        g.sayHello();
    }
}
```

```
875b8bb324a9029103fc22\redhat.java\jdt_ws\JAVA_PRACTICAL
Inner class accessing: Hello from Outer class
Hello from Anonymous class
PS C:\Users\abell\OneDrive\Desktop\JAVA_PRACTICALS>
```

Conclusion:

Inner and anonymous classes enhance Java's object-oriented capabilities by enabling tighter encapsulation and concise, one-time implementations.

Exercise:

1. Create a Vehicle program where an inner class displays vehicle details and anonymous class performs an action.

```
class Vehicle {  
  
    private String name;  
    private String type;  
    private int speed;  
  
    Vehicle(String n, String t, int s) {  
        this.name = n;  
        this.type = t;  
        this.speed = s;  
    }  
  
    class VehicleDetails {  
        void show() {  
            System.out.println("Vehicle Name: " + name);  
            System.out.println("Type: " + type);  
            System.out.println("Top Speed: " + speed + " km/h");  
        }  
    }  
}  
  
interface Action {  
    void performAction();  
}  
  
public class JAVA_EXP_6 {  
  
    public static void main(String[] args) {  
  
        Vehicle v = new Vehicle("Ferrari 488", "Sports Car", 330);  
  
        Vehicle.VehicleDetails details = v.new VehicleDetails();  
        details.show();  
  
        Action start = new Action() {  
            public void performAction() {  
                System.out.println("Vehicle is starting the engine and moving  
forward.");  
            }  
        };  
  
        start.performAction();  
    }  
}
```

```
[Running] cd "c:\Users\M-Mamoun Albrair\OneDrive\Desktop\New folder (4)\\" && javac JAVA_EXP_6.java && java JAVA_EXP_6
Vehicle Name: Ferrari 488
Type: Sports Car
Top Speed: 330 km/h
Vehicle is starting the engine and moving forward.

[Done] exited with code=0 in 12.838 seconds
```

2. Develop a Food Delivery Application where an inner class handles order details and anonymous classes handle delivery status updates.

```
class FoodOrder {

    private String customerName;
    private String foodItem;
    private double price;

    FoodOrder(String customerName, String foodItem, double price) {
        this.customerName = customerName;
        this.foodItem = foodItem;
        this.price = price;
    }

    class OrderDetails {
        void showOrder() {
            System.out.println("Customer: " + customerName);
            System.out.println("Food Item: " + foodItem);
            System.out.println("Price: Rs. " + price);
        }
    }
}

interface DeliveryStatus {
    void updateStatus();
}

public class JAVA_EXP_6 {

    public static void main(String[] args) {

        FoodOrder order = new FoodOrder("bkor", "pani puri", 250);

        FoodOrder.OrderDetails orderDetails = order.new OrderDetails();
        orderDetails.showOrder();

        DeliveryStatus preparing = new DeliveryStatus() {
            public void updateStatus() {
                System.out.println("Status: Order is being prepared");
            }
        };
        preparing.updateStatus();

        DeliveryStatus outForDelivery = new DeliveryStatus() {
```

```

        public void updateStatus() {
            System.out.println("Status: Order is out for delivery");
        }
    };
    outForDelivery.updateStatus();

    DeliveryStatus delivered = new DeliveryStatus() {
        public void updateStatus() {
            System.out.println("Status: Order has been delivered");
        }
    };
    delivered.updateStatus();
}
}

```

```

[Running] cd "c:\Users\M-Mamoun Albrair\OneDrive\Desktop\New folder (4)\\" && javac JAVA_EXP_6.java && java JAVA_EXP_6
Customer: bkor
Food Item: pani puri
Price: Rs. 250.0
Status: Order is being prepared
Status: Order is out for delivery
Status: Order has been delivered

```

